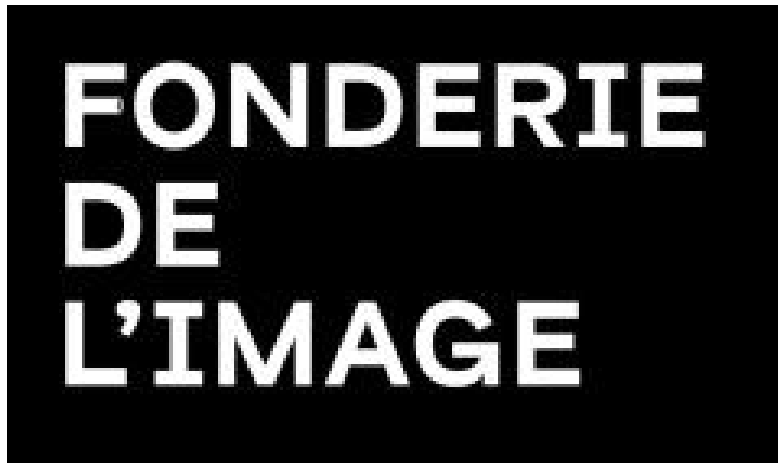




RAPPORT TECHNIQUE

Mise en Place et Optimisation d'une Infrastructure Data sur le Cloud

Collecte et stockage de données avec Python

Auteur : Lyna Lasla

Encadreur : M. Antoine Lucsko

Classe : AIA01

Date : Mai 2026

1. Introduction et contexte

Dans un contexte où les données issues du marché de l'emploi sont massives et hétérogènes, il est nécessaire de mettre en place des architectures capables de les collecter, les structurer et les exploiter efficacement.

L'objectif de ce projet est de concevoir un pipeline ETL complet permettant :

- la collecte automatisée de données depuis des API externes,
- leur nettoyage et transformation,
- leur stockage dans une infrastructure cloud,
- leur visualisation via un dashboard interactif.

Ce projet s'inscrit dans une logique de Data Engineering moderne, proche des standards industriels.

2. Architecture globale du système

L'architecture mise en place repose sur une séparation en plusieurs couches fonctionnelles :

- **Sources de données** : API Adzuna et API Remote
- **Pipeline ETL** : scripts Python d'extraction, transformation et chargement
- **Conteneurisation** : Docker et Docker Compose pour l'orchestration
- **Stockage Cloud** :
 - AWS S3 (Data Lake)
 - AWS RDS PostgreSQL (Data Warehouse)
- **Visualisation** : dashboard interactif développé avec Plotly et technologies web

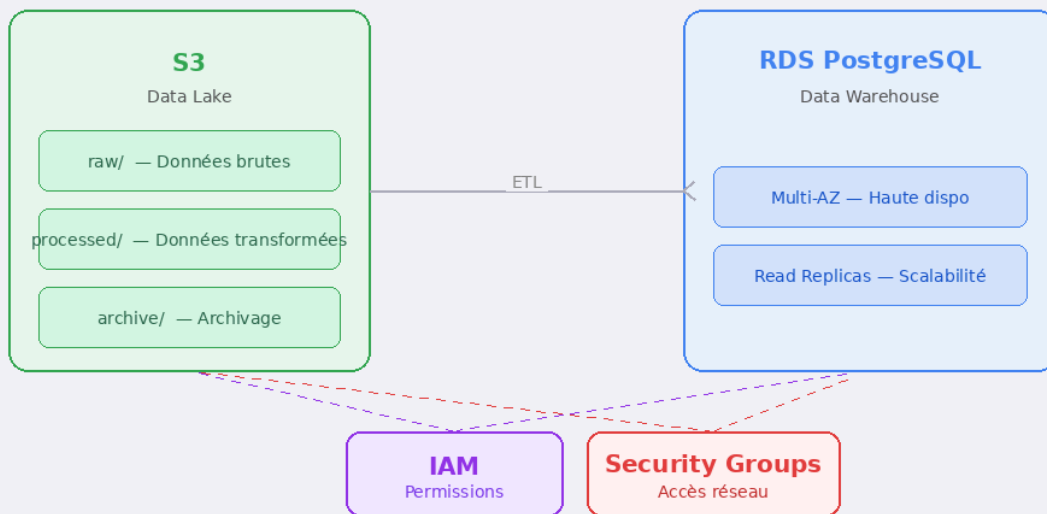
3. Architecture Cloud AWS

L'infrastructure cloud repose sur plusieurs services AWS :

- **AWS S3** : stockage des données brutes et transformées (Data Lake)
- **AWS RDS PostgreSQL** : base relationnelle utilisée comme Data Warehouse
- **IAM** : gestion des accès et des permissions
- **Security Groups** : sécurisation des communications réseau

Cette architecture permet de séparer clairement le stockage brut et les données exploitables.

Architecture Cloud AWS



4. Conteneurisation et orchestration

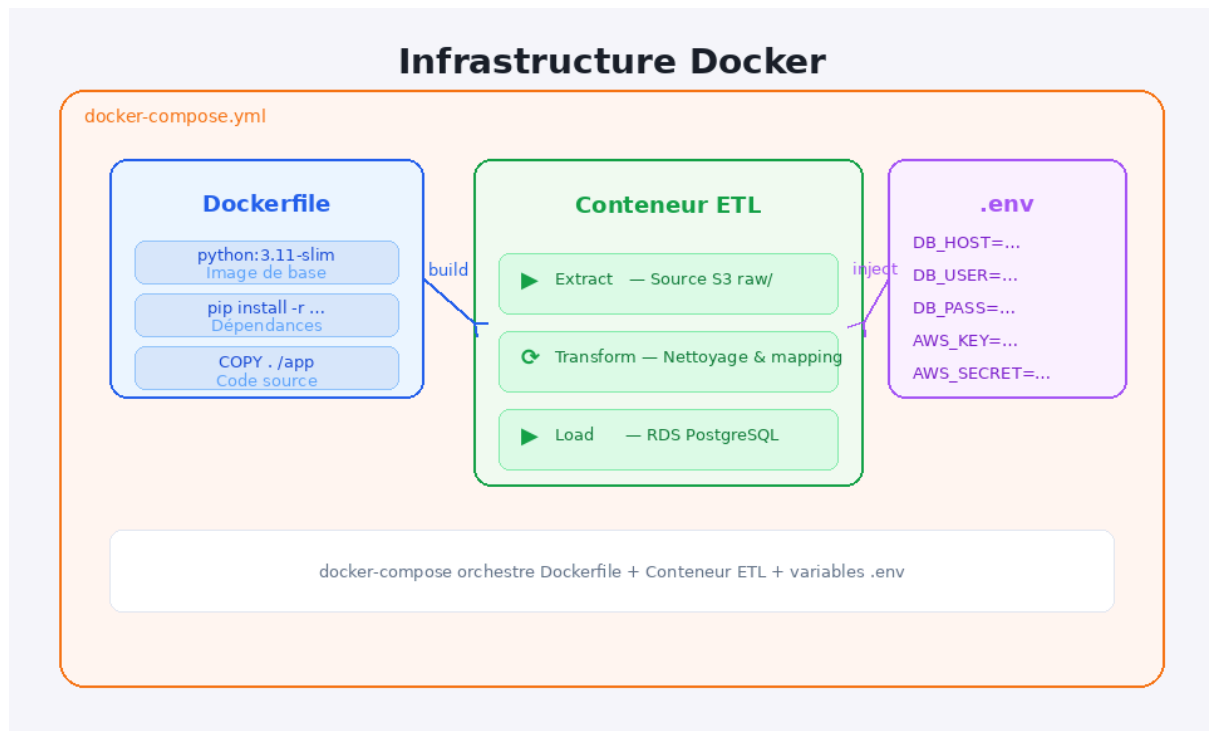
Le projet est conteneurisé à l'aide de Docker afin de garantir :

- la reproductibilité de l'environnement,
- la portabilité du projet,
- l'isolation des dépendances.

Les composants utilisés sont :

- un **Dockerfile** définissant l'environnement Python,
- un fichier **docker-compose.yml** orchestrant l'exécution des services,
- des variables d'environnement centralisées dans un fichier **.env**.

Le conteneur exécute automatiquement les étapes du pipeline ETL.



5. Pipeline ETL

Le pipeline est structuré en trois étapes principales :

5.1 Extraction (Extract)

Les données sont collectées à partir de deux sources :

- API Adzuna
- API Remotive

Les scripts responsables sont :

- `scripts/api.py`
- `scripts/scrape.py`

5.2 Transformation (Transform)

Les données sont traitées à l'aide de Pandas :

- fusion des sources
- suppression des doublons
- normalisation des intitulés de postes
- enrichissement des données (niveau d'expérience, longueur des titres)
- nettoyage des valeurs manquantes

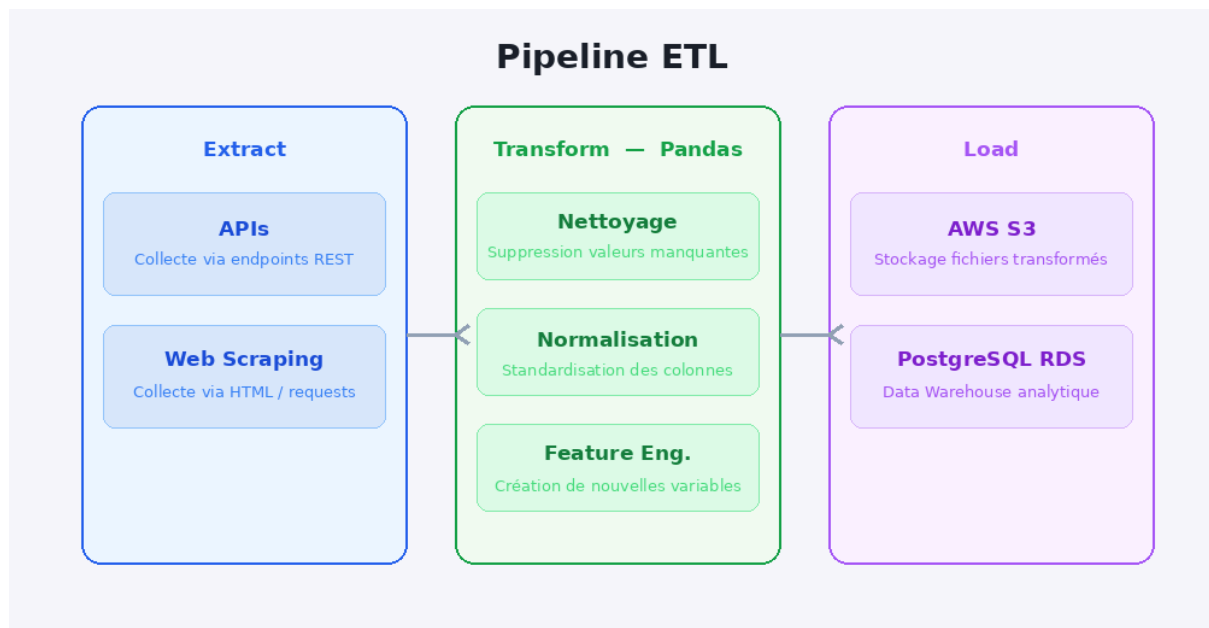
5.3 Chargement (Load)

Les données transformées sont chargées vers deux destinations :

- **AWS S3** : stockage des fichiers CSV (Data Lake)
- **AWS RDS PostgreSQL** : stockage structuré pour analyse

Scripts utilisés :

- `etl/to_s3.py`
- `etl/load_to_rds.py`



6. Analyse des données

Plusieurs analyses exploratoires ont été réalisées :

- identification des entreprises les plus recruteuses,
- analyse géographique des offres,
- classification des niveaux de postes (junior, mid, senior),
- extraction des compétences les plus demandées,
- comparaison des sources de données.

Ces analyses permettent une meilleure compréhension du marché de l'emploi dans le domaine de la Data.

7. Dashboard interactif

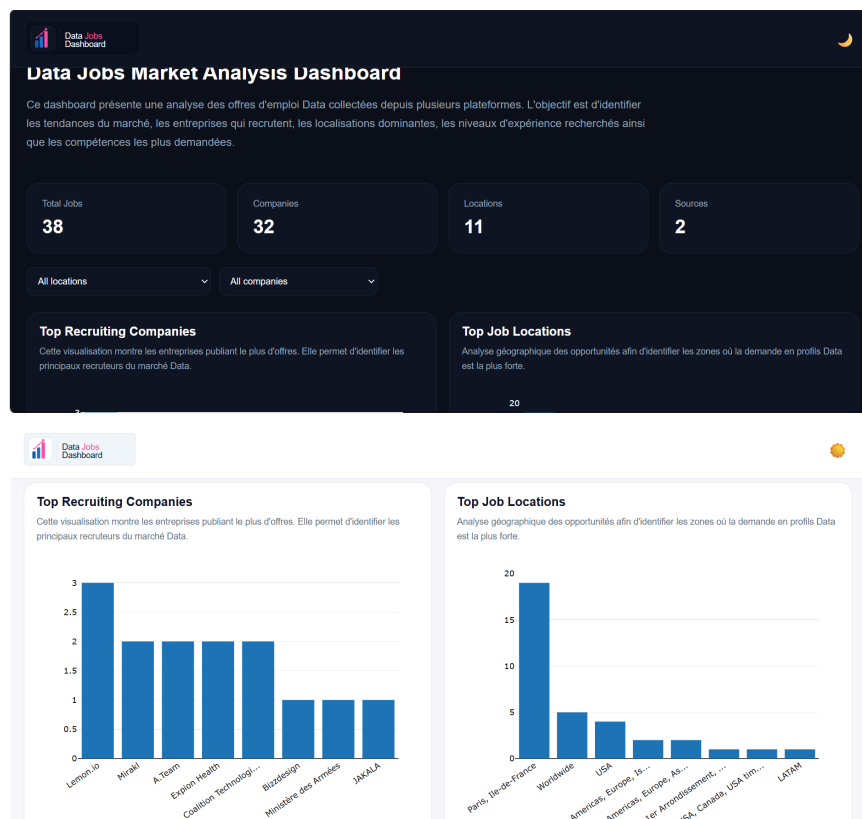
Un dashboard interactif a été développé afin de visualiser les résultats.

Fonctionnalités principales :

- indicateurs clés de performance (KPIs),
- filtres dynamiques (localisation, entreprise),
- graphiques interactifs (barres, histogrammes, camemberts),
- analyse des compétences,
- interface responsive avec mode clair/sombre.

Technologies utilisées :

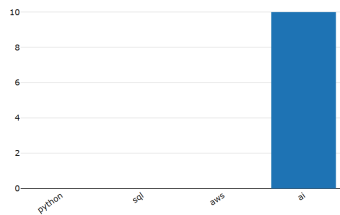
- Plotly
- HTML / CSS / JavaScript





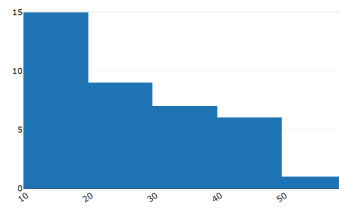
Skills Detection

Détection des compétences mentionnées dans les titres des offres afin d'identifier les technologies les plus visibles.



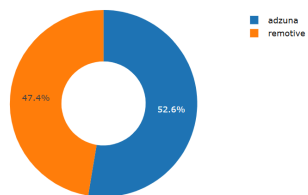
Job Title Complexity

Distribution de la longueur des intitulés de poste. Cette analyse aide à comprendre la complexité et la spécialisation des rôles proposés.



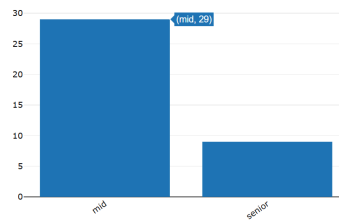
Data Sources Distribution

Répartition des offres selon les plateformes utilisées pour la collecte des données.



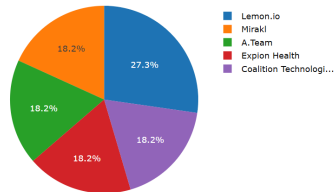
Experience Levels

Comparaison des niveaux d'expérience demandés dans les offres : junior, mid et senior.



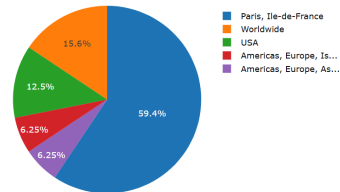
Top Companies Share

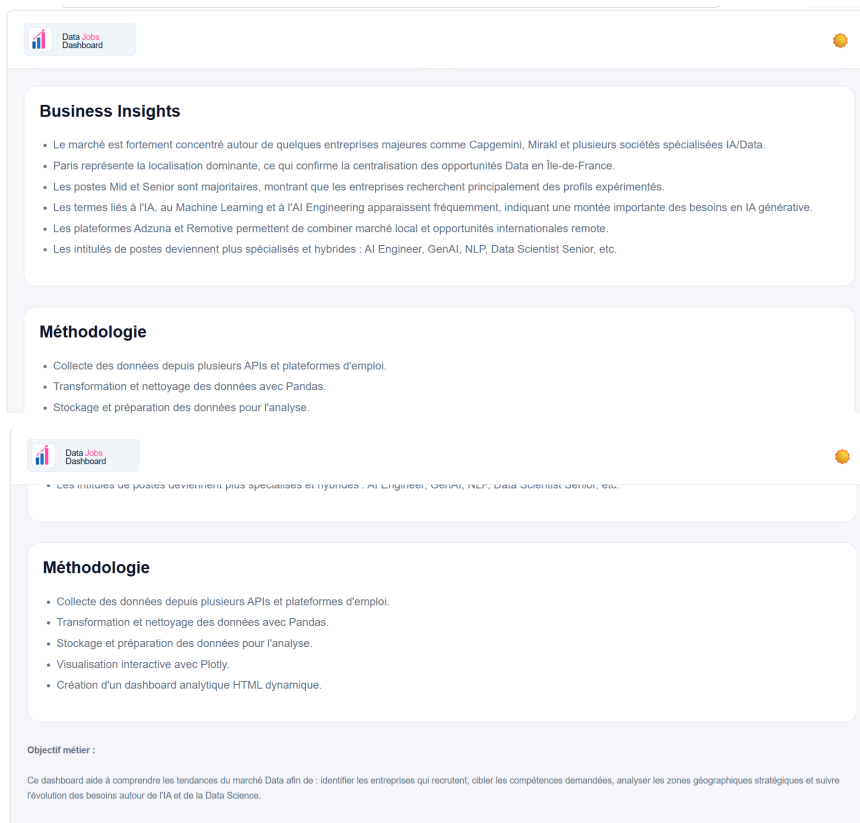
Part des principales entreprises dans l'ensemble des offres collectées.



Top Locations Share

Répartition des offres entre les principales zones géographiques.





8. Organisation du projet

L'architecture logicielle est structurée de manière modulaire :

```

projet-data/
├── aws/           (configuration cloud)
├── dashboard/     (interface de visualisation)
├── data/          (datasets bruts et traités)
├── etl/           (chargement vers cloud)
├── scripts/       (extraction et transformation)
├── notebooks/     (analyse exploratoire)
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── .env
  
```

Cette organisation garantit :

- la lisibilité du code,
- la maintenabilité,
- la séparation des responsabilités.

9. Sécurité et bonnes pratiques

Plusieurs mesures de sécurité ont été mises en place :

- utilisation de variables d'environnement pour les credentials,
- gestion des accès via IAM (principe du moindre privilège),
- sécurisation des accès réseau via Security Groups,
- exclusion des données sensibles du dépôt Git,
- isolation des services via Docker.

10. Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées :

- configuration des accès AWS (IAM et S3),
- connexion sécurisée à PostgreSQL RDS,
- orchestration Docker du pipeline,
- gestion de données hétérogènes issues de plusieurs API,
- nettoyage et normalisation des données textuelles.

Ces difficultés ont permis de renforcer les compétences en Data Engineering et Cloud Computing.

11. Conclusion

Ce projet a permis de concevoir une architecture Data Cloud complète intégrant l'ensemble des étapes d'un pipeline de Data Engineering : ingestion, transformation, stockage et visualisation.

Il reproduit des pratiques professionnelles utilisées en entreprise et met en œuvre des technologies modernes telles que AWS, Docker et Python.

12. Perspectives d'amélioration

Les évolutions possibles du projet sont :

- mise en place d'Apache Airflow pour l'orchestration,
- automatisation CI/CD via GitHub Actions,
- streaming de données en temps réel (Kafka ou AWS Kinesis),
- intégration de modèles de Machine Learning (prédiction de salaires, recommandations),
- monitoring avancé via AWS CloudWatch.

